

Architecting Visual Multi-Agent Swarms: Desktop-First Systems Engineering with the Cursor SDK

Distributed Topologies and Multi-Agent Orchestration Theory

The transition from single-prompt conversational artificial intelligence to autonomous multi-agent engineering collectives represents a structural inflection point in software automation¹. Monolithic conversational sessions inevitably suffer from context contamination, token saturation, and non-deterministic task execution when tasked with multi-stage software delivery¹. As an agent's context window fills with intermediate reasoning steps, raw terminal outputs, and repeated codebase traversals, the underlying large language model experiences significant cognitive degradation, manifesting as lost instruction adherence and hallucinated syntax¹. A visual multi-agent swarm platform resolves these constraints by providing an interactive spatial canvas where users compose, configure, and inspect directed workflows of autonomous, decoupled agents³.

The fundamental architecture decouples high-level cognitive orchestration from low-level workspace execution². The visual orchestration canvas manages agent topologies, parameter assignments, data dependencies, and human steering boundaries³. Beneath this visual abstraction, a centralized orchestration engine compiles the node graph into an executable Directed Acyclic Graph (DAG), manages asynchronous task scheduling, enforces transactional state persistence, and arbitrates inter-agent dependencies². At the physical execution boundary, the system leverages an agent execution harness to provision isolated file workspaces, attach Model Context Protocol (MCP) tool servers, run terminal commands, and stream execution telemetry back to the user interface¹.

Coordination Topology	Structural Mechanics	State & Message Routing	Failure Domain Isolation	Primary Engineering Use Cases
Topological DAG Runner	Directed Acyclic Graph; strict dependency tiers ²	Deterministic edge-based data handoffs; upstream artifacts feed downstream prompts ²	High; tasks run in parallel only when unblocked; zero write collisions on shared files ²	Codebase refactoring, multi-package builds, CI/CD remediation pipelines ²

Hierarchical Supervisor	Centralized tree (Root Planner to Leaf Workers) ¹	Planner assigns scoped manifests; workers return structured summaries ¹	High; workers operate in isolated sandboxes; supervisor aggregates progress ¹	Full-stack feature delivery requiring spec drafting, backend, and frontend sync ¹
Decentralized Actor Mesh	Fully connected peer-to-peer network ¹	Asynchronous message passing via mailboxes or shared event queues ¹	Low; elevated risk of conversational loops, token exhaustion, and conflicting workspace mutations ²	Open-ended architectural debates, red-teaming, multi-perspective code reviews ²

For software engineering swarms, combining a topological DAG task runner with hierarchical worker isolation provides the greatest operational reliability¹. Unstructured peer-to-peer communication models rapidly exhaust token budgets and frequently enter recursive conversational loops where agents debate implementation details without committing changes². In contrast, modeling agent interactions as an explicit dependency graph ensures that operations such as database schema migrations, backend API scaffolding, client generation, and integration testing execute in strict chronological sequence while fanning out independent subtasks in parallel².

When multiple agents collaborate on a shared issue within a single graph stage, the orchestration engine enforces a contract-first synchronization phase⁴. Before parallel executors write code, an upstream planner or supervisor freezes API contracts, shared type definitions, and interface stubs⁴. Downstream workers write implementation code against these static contracts using mock interfaces⁴. Furthermore, agents are constrained via Abstract Syntax Tree (AST) scoping, restricting each worker's write capabilities to specific directories or packages while allowing read-only codebase traversal for context⁴.

Visual Canvas and Node-Graph Engineering

The visual editor represents the primary operational interface for workflow composition. Building a fluid, Miro-like canvas requires handling arbitrary node positioning, responsive panning and zooming, custom node definitions with multiple dynamic connection ports (handles), interactive edge routing, and real-time execution feedback³.

UI Engine /	Core Rendering	Reactivity & State	Custom Node	Dynamic Handle	Rendering	Cross-Platform
-------------	----------------	--------------------	-------------	----------------	-----------	----------------

Canvas Library	g Architecture	Integration	Extensibility	Support	Performance Under Load	Viability
React Flow (@xyflow/react)	DOM/SVG Hybrid with @xyflow/system [cite: 3]	Native React Hooks; seamless Zustand store binding ³	Declarative JSX components; native HTML inputs and forms ³	High; dynamic IDs managed via useUpdateNodeInternals [cite: 3, 5]	Spatial indexing; smooth 60 FPS under 1,000+ nodes ³	Universal across Web, Electron, and Tauri webviews ³
Rete.js (v2)	Modular plugin architecture (framework-agnostic) ³	Custom reactive event bus; adapters for React/Vue ³	Requires custom render plugins and socket abstractions ³	Moderate; programmatic socket creation ³	High; canvas or DOM-based depending on render plugin ³	Universal across Web, Electron, and Tauri ³
LiteGraph.js	Pure HTML5 2D Canvas ³	Imperative callback graph; monolithic state ³	Low; manual drawing via 2D Canvas context commands ³	High programmatic support; rigid visual layout ³	Extreme (10,000+ nodes; ComfyUI backend) ³	High; lacks native DOM styling for inputs/forms ³
Flutter Canvas (flutter_flow_chart)	Skia / Impeller native rendering engine ³	Flutter reactive widget tree; Riverpod / Bloc ³	High within Dart; isolated from web ecosystem ³	Moderate; requires manual hit-testing implementations ³	High desktop native GPU execution ³	Desktop native; requires total rewrite of web tooling ³

React Flow (@xyflow/react) provides the optimal foundation for this architecture³. Because agent configuration requires rich interactive controls—including multi-line system prompt

textareas, searchable MCP server dropdowns, token consumption badges, and file drag-and-drop zones—rendering these nodes with native HTML and JSX avoids the substantial engineering overhead of building custom hit-testing, text selection, and form handling inside raw canvas libraries like LiteGraph³.

Dynamic handles are essential for multi-agent workflows³. An agent node must dynamically surface input handles for upstream agent outputs, dropped file contexts, and MCP tool connections, alongside output handles for code diffs, structured handoff summaries, and execution triggers². When handles are added or removed dynamically (for instance, when a user attaches an external database or adds a new context file), React Flow requires an explicit call to the useUpdateNodeInternals(nodexd) hook³. This recomputes the DOM bounding client rectangles of the connection ports and recalibrates the SVG path calculation of connected edges³.

To deliver clear visual feedback, edge states reflect real-time execution in the orchestration engine³. A custom edge component tracks node execution state from a shared Zustand store³. Idle edges render with a muted gray bezier stroke (#475569, 1.5px)³. Running edges transition to a glowing cyan stroke (#06b6d4, 2.5px) utilizing an SVG animation that moves a glowing particle or gradient along the calculated path using the Web Animations API, avoiding the severe CPU and GPU performance bottlenecks associated with mutating stroke-dasharray across large diagrams³. Failed or blocked states transition immediately to a pulsing crimson stroke (#ef4444, 2px) indicating unfulfilled dependencies or syntax errors³.

Click-to-connect interactions are implemented using React Flow's onConnect and isValidConnection callbacks⁸. When a user drags a connection line from an upstream output handle to a downstream input handle, the system validates the type compatibility between handles (such as ensuring an MCP Tool output connects only to an Agent Tool input, or a Git Diff output connects to a Code Review input) before creating the edge record in the Zustand store³.

Desktop Shell Architecture and Windows-First Systems Integration

Developing a desktop application on Windows while ensuring clean future expansion to macOS and Linux requires selecting a shell that balances local operating system access with packaging simplicity³.

Architectural Parameter	Electron (Chromium + Node.js)	Tauri v2 (Rust Core + System WebView)	Browser-First Web App + Local Daemon
Target OS Rendering Engine	Bundled Chromium (identical across all OS platforms) ³	Windows 10/11 Evergreen WebView2	Host Browser (Edge, Chrome, Firefox) ³

		(Chromium-based) ³	
Desktop Installer Footprint	80 MB–140 MB installer (NSIS / MSI) ³	5 MB–15 MB installer (NSIS / MSIX) ³	Zero client download (Web) + ~40 MB backend binary ³
Idle Memory Consumption	120 MB–250 MB RAM ³	25 MB–50 MB RAM ³	Browser tab overhead + server background process ³
Cursor SDK Runtime Integration	Native in-process execution inside Node.js background process ³	Requires Rust sidecar process or local Node/Bun daemon ³	Executed inside backend daemon process ³
Windows Build Friction	Very Low; standard npm run build with electron-builder [cite: 3]	Moderate; requires MSVC C++ Build Tools & Rust toolchain ³	Very Low; pure web stack and backend scripts ³
Cross-Platform Portability	Write once, package identically across Windows, macOS, Linux ³	High; requires testing against WebKitGTK on Linux ³	Universal across any operating system with a browser ³

Electron is the definitive runtime choice for Windows-first implementation³. This recommendation is dictated by language runtime alignment: the official, feature-complete Cursor SDK (@cursor/sdk) is a TypeScript package targeting Node.js 22.13 or later³. In Electron, @cursor/sdk executes directly inside a dedicated Node.js background process (UtilityProcess or Main Process) without IPC serialization friction or external interpreter dependencies³. If Tauri v2 were selected, invoking the Cursor TypeScript SDK would require packaging a separate Node.js or Bun runtime as a sidecar binary, introducing inter-process communication overhead and doubling packaging complexity on Windows³.

Operating on Windows introduces specific filesystem and process lifecycle challenges that dictate defensive systems programming³. The Windows NT kernel enforces mandatory file locking whenever a process holds an open file descriptor, contrasting sharply with POSIX systems where files and directories can be unlinked while active handles exist³. When an agent spawns an MCP server, background file watcher, or language server whose working directory points inside an active Git worktree, Windows locks the entire directory tree³. Any subsequent

call to `git worktree remove` throws fatal: Permission denied or leaves empty, undeletable locked directory shells on disk³.

To prevent these crashes, the orchestration engine must enforce a strict, hierarchical teardown cascade:



This sequence systematically strips handles before unlinking the directory structure³. First, the engine scans and terminates all active child process trees using `taskkill /pid <PID> /T /F`.

Second, the host Node.js process resets its working directory to the application root via `process.chdir(appRoot)` to ensure the host process does not retain locks on the workspace³.

Third, an escalating backoff retry loop executes filesystem deletions across exponential delays (50ms, 150ms, 500ms, 2000ms), granting the Windows kernel time to flush lingering file handles before `git worktree remove --force` executes³.

In addition to locking semantics, deeply nested workspace directories—such as `node_modules` inside nested agent checkouts—frequently exceed the classic Win32 260-character path limit (`MAX_PATH`)³. The desktop application manifest must explicitly declare `<longPathAware>true</longPathAware>`, and the internal runtime must format absolute Windows paths using the extended-length syntax prefix (`\\?\\C:\\path\\...`) when performing directory cleanup operations³.

Furthermore, terminal execution on Windows demands pseudo-terminal (PTY) allocation. Executing background tasks or unit tests through standard subprocess pipes causes child processes to buffer output unpredictably or hang indefinitely when interactive prompts are encountered. Local tools and verification runners executed by agents must leverage Windows ConPTY via `node-pty`, compiled against the Electron ABI using modern C++ build tools rather than legacy wrappers.

Cursor SDK Architectural Evaluation: Resolving the TypeScript vs. Python Dilemma

While preliminary surveys frequently assume Python is the natural choice for multi-agent workflows, an architectural analysis reveals that the Cursor TypeScript SDK (@cursor/sdk) is decisively superior to the Cursor Python SDK (cursor-sdk) for building this visual desktop platform³.

Evaluation Vector	Cursor TypeScript SDK (@cursor/sdk)	Cursor Python SDK (cursor-sdk)	Architectural Impact on Desktop App
Package Maturity & Version	Production-ready (v1.0.x) with rapid update cycles ³	Early Beta (v0.1.x) tracking bridge contract updates ³	TypeScript receives upstream capabilities and fixes first ³
Underlying Architecture	Native in-process Node.js harness; embeds agent runtime ³	Thin client wrapper communicating via Connect RPC to cursor-sdk-bridge [cite: 3, 17]	Python introduces an extra RPC layer and binary dependency ³
System Prompt Overrides	First-class systemPrompt option on Agent.create() and Agent.resume() [cite: 3, 5]	Not directly exposed in base options in v0.1.x (requires prompt prefixing) ³	Essential for defining custom agent personas and rigid output contracts ³
Dynamic In-Flight Steering	Native run.steer(text) method to inject steering mid-execution ³	Not supported in v0.1.x; binary execution flow only (wait or cancel) ³	Enables the operator to guide the agent without aborting execution ³
Execution Cancellation	Deterministic via run.cancel() across local and cloud runtimes ³	Supported via run.cancel() [cite: 3, 18]	Immediate interruption of runaway loops and token drain ³
Desktop Shell	Zero overhead;	High; requires	TypeScript

Alignment	executes directly in Electron's Node.js engine ³	bundling Python interpreter and virtualenvs on Windows ³	maintains a single language runtime across canvas and engine ³
------------------	---	---	---

The technical imperative for selecting TypeScript centers on runtime architecture, feature availability, and systems overhead³. First, the TypeScript SDK exposes the `systemPrompt` property on `Agent.create()` and `Agent.resume()`, completely replacing Cursor's built-in system prompt for the main agent³. This capability allows the platform to strip standard conversational behaviors and inject rigid, machine-readable JSON schemas and operational boundaries directly into the agent reasoning loop⁵. In the Python v0.1.x SDK, system prompt overrides are not exposed as first-class constructor options, forcing orchestrators to prefix user prompts with instructions, which degrades model attention³.

Second, dynamic in-flight steering is natively supported in TypeScript via `run.steer(text)`³. If an operator observes that an agent is refactoring an incorrect module or pursuing an invalid architectural path while inspecting real-time streaming tokens, the operator can inject corrective instructions into the active turn without cancelling the run or losing conversation state³. The Python SDK lacks dynamic steering, offering only a binary wait-or-cancel execution control³.

Third, the Python SDK operates as a thin client communicating over Connect RPC to `cursor-sdk-bridge`—a local background daemon that embeds the TypeScript SDK³. Selecting Python would introduce a convoluted three-tier process topology consisting of the Electron frontend, a local Python interpreter, and the SDK bridge daemon³.

Fourth, selecting `@cursor/sdk` establishes complete language uniformity across the technology stack³. The React Flow UI, Zustand state store, Electron main process, DAG scheduler, and Cursor execution engine all share a single TypeScript runtime³. Bundling an embedded Python interpreter, managing virtual environments, and resolving native wheel dependencies on Windows introduces severe packaging bloat and end-user friction that is avoided by adopting TypeScript³.

Below is the concrete implementation of an execution worker utilizing `@cursor/sdk`:

```
TypeScript
import Agent, type SDKMessage from '@cursor/sdk'

export interface WorkerExecutionConfig {
  agentId: string
  systemPrompt: string
  taskPrompt: string
}
```

```

contextData string
workspaceDir string
apiKey string
onStreamChunk (chunk: SDKMessage) => void

```

```

export class AgentExecutionWorker
private activeAgent: Agent | null = null
private activeRun: ReturnType<Agent['send']> | null = null
private isAborted: boolean = false

```

```

public async execute(config: WorkerExecutionConfig): Promise<string> {
  if (this.isAborted) {
    throw new Error('Execution aborted prior to launch.')
  }

```

```

  this.activeAgent = await Agent.create({
    apiKey: config.apiKey,
    model: { id: 'composer-2.5' },
    systemPrompt: config.systemPrompt,
    local: { cwd: config.workspaceDir },
  })

```

```

  try {
    const fullPrompt = `### CONTEXT DATA:\n${config.contextData}\n\n###
OBJECTIVE:\n${config.taskPrompt}`
    this.activeRun = this.activeAgent.send(fullPrompt)

```

```

    let finalOutputText = ""

```

```

    for await (const message of this.activeRun) {
      config.onStreamChunk(message)

```

```

      if (message.type === 'assistant') {
        for (const block of message.content) {
          if (block.type === 'text') {
            finalOutputText += block.text
          }
        }
      }
    }

```

```

    const outcome = await this.activeRun.result

```

```

    if (outcome.status === 'failed')
        throw new Error `Agent run failed: ${outcome.error?.message ?? 'Unknown error'}`
    }

    return finalOutputText
    finally
        await this.activeAgent?.Symbol.asyncDispose()
        this.activeAgent = null
        this.activeRun = null
    }
}

public async steer(guidanceText: string): Promise<void>
{
    if (this.activeRun && typeof this.activeRun.steer === 'function')
        await this.activeRun.steer(guidanceText)
    }
}

public async abort(): Promise<void>
{
    this.isAborted = true
    if (this.activeRun)
        await this.activeRun.cancel()
    }
}

```

Multi-Modal Ingestion, Context Propagation, and MCP Architecture

Passing unedited, multi-turn conversational transcripts across agent boundaries is an architectural anti-pattern that leads directly to context saturation and execution drift¹. If a downstream validation or testing agent receives the raw reasoning traces, compilation errors, and intermediate code revisions of an upstream development agent, the downstream model's reasoning capacity is diluted by irrelevant operational noise¹.

To preserve clean context boundaries, the platform enforces structured artifact handoffs¹. Upstream agents synthesize their deliverables into standardized schemas before data is passed across visual edges². These handoffs typically include a summary of completed requirements, a list of modified files, exact interface specifications, and active blockers². Downstream agents ingest this structured summary via prompt templates that inject the upstream artifact directly into their initial context window².

Input Modality	Canvas Representation	Ingestion & Parsing Mechanism	Injected Agent Format
User File Drop	Visual File Drop Node on Canvas	HTML5 drag-and-drop listener; copies files to project run cache (.inputs/)	Inlined as formatted Markdown code blocks or attached via SDKImage structures
Local Folder / Drive	Workspace Directory Node	Validated absolute Win32 path; sets local: { cwd: path } in agent config	Direct local filesystem access; indexed via Cursor semantic codebase search
External Database	Database Adapter Node	Local MCP Stdio bridge exposing dynamic SQL query tools	Dynamic tool calls (query_schema, execute_read_query) inside agent loop
External MCP Server	MCP Service Node	Stdio child process or HTTP/SSE remote client connection	Registered dynamically in Agent.create({ mcpServers }) configuration
Upstream Agent Output	Directed Canvas Edge	Output extraction hook parsing git diffs or JSON handoff summaries	Templated context variables injected into downstream agent prompt

The Model Context Protocol (MCP) establishes an open standard for integrating tools and shared state¹. Instead of embedding bespoke database drivers into the orchestration engine, external databases and services run as local or remote MCP servers¹. When a user connects an MCP Node to an Agent Node, the orchestrator registers the server definition dynamically in the agent options². This allows the agent to inspect database schemas, query tables, or interact with external tracking systems using standardized tool calls without bloating the core application codebase¹.

Below is the implementation of an MCP Stdio connection bridge using @modelcontextprotocol/sdk:

TypeScript

```
import Client from '@modelcontextprotocol/sdk/client/index.js'  
import StdioClientTransport from '@modelcontextprotocol/sdk/client/stdio.js'
```

```
export interface MCPServerDefinition
```

```
  id: string  
  name: string  
  command: string  
  args: string  
  env?: Record<string, string>  
}
```

```
export class MCPBridgeManager
```

```
  private activeClients = new Map<string, Client>()
```

```
  public async connectServer(def: MCPServerDefinition): Promise<Client> {
```

```
    if (this.activeClients.has(def.id))
```

```
      return this.activeClients.get(def.id)!
```

```
  }
```

```
    const transport = new StdioClientTransport({
```

```
      command: def.command,
```

```
      args: def.args,
```

```
      env: [...process.env, ...(def.env ?? {})]
```

```
    })
```

```
    const client = new Client({
```

```
      name: 'swarm-desktop-host' version '1.0.0'
```

```
      capabilities: { tools, resources }
```

```
    })
```

```
    await client.connect(transport)
```

```
    this.activeClients.set(def.id, client)
```

```
    return client
```

```
  }
```

```
  public async getAvailableTools(serverId: string)
```

```
  {  
    const client = this.activeClients.get(serverId)
```

```
    if (!client) {
```

```

        throw new Error `MCP Server ${serverId} is not connected.`
    }

    return await client.listTools()
}

public async disconnectAll(): Promise<void> {
    for (const {id, client} of this.activeClients.entries()) {
        await client.close()
        this.activeClients.delete(id)
    }
}

```

Graph Schedulers, State Persistence, and Portable Schemas

Autonomous agents cannot be coordinated through unconstrained event loops without risking deadlocks or infinite execution cycles². Deterministic workflow orchestration requires compiling the visual node-graph into a topological Directed Acyclic Graph (DAG) task engine².

The orchestration engine models nodes as vertices V and handle connections as directed edges E ³. The engine calculates the in-degree (number of incoming dependencies) for each node using Kahn's algorithm, establishing execution tiers with computational complexity $\mathcal{O}(|V| + |E|)$ ³.

```

TypeScript
export interface GraphNode {
    id: string
    type: 'agent' | 'fileInput' | 'mcp' | 'approval'
    status: 'idle' | 'queued' | 'running' | 'completed' | 'failed' | 'cancelled'
    data: Record<string, any>
}

export interface GraphEdge {
    id: string
    source: string
    target: string
}

```

```

export class DAGEngine
{
    public static calculateExecutionTiers(nodes: GraphNode[], edges: GraphEdge[]): string[] {
        const inDegree = new Map<string, number>();
        const adjacency = new Map<string, string[]>();

        nodes.forEach((node) => {
            inDegree.set(node.id, 0);
            adjacency.set(node.id, []);
        });

        edges.forEach((edge) => {
            inDegree.set(edge.target, (inDegree.get(edge.target) ?? 0) + 1);
            adjacency.get(edge.source)?.push(edge.target);
        });

        const tiers: string[] = [];
        let currentTier = nodes.filter((n) => inDegree.get(n.id) === 0).map((n) => n.id);

        let processedCount = 0;

        while (currentTier.length > 0) {
            tiers.push(currentTier);
            processedCount += currentTier.length;

            const nextTier: string[] = [];

            for (const nodeId of currentTier) {
                const neighbors = adjacency.get(nodeId) ?? [];
                for (const neighbor of neighbors) {
                    const updatedDegree = (inDegree.get(neighbor) ?? 1) - 1;
                    inDegree.set(neighbor, updatedDegree);
                    if (updatedDegree === 0) {
                        nextTier.push(neighbor);
                    }
                }
            }
            currentTier = nextTier;
        }

        if (processedCount !== nodes.length) {
            throw new Error('Cycle detected in workflow graph: topological execution is impossible.');
        }
    }
}

```

```
return
```

The execution lifecycle transitions through strictly governed states within the Zustand store. Nodes initialize in an idle state. When execution begins, root nodes transition to queued and enter running status within an asynchronous worker pool. Downstream nodes remain idle until all parent dependencies transition to completed status. If any agent encounters an unrecoverable failure, its status shifts to failed, immediately halting downstream execution along its dependent branch while permitting unrelated branches to continue execution. When an operator triggers a cancellation, the engine invokes `await run.cancel()`, terminating the underlying network stream, killing child processes, and persisting the intermediate state to disk. To ensure crash resilience and multi-session continuity, execution states are persisted locally using SQLite via `better-sqlite3`³. SQLite provides zero-configuration durability and ACID compliance without the operational overhead of an external database server¹. Write-Ahead Logging (WAL) mode is enabled to support non-blocking concurrent reads while agent execution runs stream continuous telemetry writes³.

```
SQL
PRAGMA journal_mode = WAL
PRAGMA foreign_keys = ON

CREATE TABLE IF NOT EXISTS workflows (
  id TEXT PRIMARY KEY
  name TEXT NOT NULL
  description TEXT
  schema_version TEXT NOT NULL
  graph_topology TEXT NOT NULL
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
)
```

```
CREATE TABLE IF NOT EXISTS execution_runs (
  id TEXT PRIMARY KEY
  workflow_id TEXT NOT NULL
  status TEXT NOT NULL
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  completed_at DATETIME
)
```

```

FOREIGN KEY (workflow_id) REFERENCES workflows(id) ON DELETE CASCADE;

CREATE TABLE IF NOT EXISTS node_checkpoints (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL,
  node_id TEXT NOT NULL,
  status TEXT NOT NULL,
  system_prompt_snapshot TEXT NOT NULL,
  input_payload TEXT,
  output_payload TEXT,
  workspace_branch TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (run_id) REFERENCES execution_runs(id) ON DELETE CASCADE;

```

Workflows are serialized into an open JSON schema that defines viewport geometry, node configurations, system prompts, handle mappings, and environment variable requirements³. When exporting workflows for external collaborators, all operational secrets (API keys, personal tokens, private database connection strings) are stripped from node configurations³. The schema declares these dependencies in `environment.requiredEnvVars`³. The workflow JSON is packaged into a compressed `.swarm` archive alongside referenced prompt templates and seed assets³. On import, the platform validates schema integrity, confirms graph acyclicity, prompts the importing user to supply the missing environment variables, and registers the workflow in the local database³.

```

JSON
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "version": "1.0.0",
  "metadata": {
    "id": "wf-8f92-41a2",
    "name": "Full-Stack Feature Swarm",
    "description": "Decomposes feature specs, builds backend and frontend concurrently, runs validation.",
    "createdAt": "2026-10-03T08:00:00Z"
  },
  "environment": {
    "requiredEnvVars": ["CURSOR_API_KEY", "DATABASE_URL"]
  }
}

```

```
"graph"
  "viewport" ["x" 120.5 "y" 84.0 "zoom" 1.15]
  "nodes"
    "id" "node-planner-1"
    "type" "agent"
    "position" ["x" 100 "y" 250]
    "data"
      "label" "Systems Planner"
      "model" "composer-2.5"
      "systemPrompt" "You are a principal architect. Break down requirements into backend and frontend tasks."
      "inputs" ["id" "in-reqs" "name" "Requirements" "type" "file"]
      "outputs" ["id" "out-spec" "name" "Specification" "type" "string"]
    "id" "node-backend-1"
    "type" "agent"
    "position" ["x" 550 "y" 150]
    "data"
      "label" "Backend Worker"
      "model" "composer-2.5"
      "systemPrompt" "Implement API routes strictly conforming to the provided specification."
      "inputs" ["id" "in-spec" "name" "Specification" "type" "string"]
      "outputs" ["id" "out-diff" "name" "Backend Diff" "type" "diff"]
      "mcpServers" "mcp-db-server"
  "edges"
    "id" "edge-planner-to-backend"
    "source" "node-planner-1"
    "sourceHandle" "out-spec"
    "target" "node-backend-1"
    "targetHandle" "in-spec"
    "animated" true
```

Workspace Concurrency and Git Worktree Sandboxing

When multiple agents run in parallel to solve complex objectives—such as building frontend interfaces, backend endpoints, and unit test suites simultaneously—running them in a single workspace creates destructive race conditions¹.

If two autonomous agents operate concurrently within the same physical checkout, multiple silent failures occur. At the filesystem level, agents overwrite intermediate file edits, generating syntactically invalid source code or reverting concurrent improvements. At the version control layer, the Git staging index (.git/index) is shared across the entire checkout. When two parallel agents execute staging and commit commands concurrently, the index quietly merges contributions from both into an uncoordinated hybrid commit, corrupting repository history without triggering an explicit merge error. Furthermore, concurrent processes step on shared network ports, background databases, and local cache locks.

Isolation Primitive	Filesystem Disk Overhead	Process Isolation Level	Shared Object Database	Provisioning Latency	Primary Tradeoff
Shared Directory	None ¹	None (Severe race conditions) ¹	Shared (.git) ¹	Zero ¹	Corrupts staging index and causes write collisions ¹
Git Worktree	Minimal (Only checked-out tree) ¹	Branch/Directory level isolation ¹	Shared (.git) ¹	< 500 ms ¹	Requires Windows file-lock (EBUSY) defensive cleanup ³
Docker / Podman	Medium (Base OS image layers) ¹	OS Container (cgroups, namespaces) ¹	Isolated or Volume Mounted ¹	2–5 s ¹	Substantial CPU/RAM overhead on developer laptops ¹

Micro-VMs (Modal, E2B)	High (Hypervisor disk images) ¹	Complete Hardware Virtualization ¹	Fully Isolated Network/Disk ¹	5–15 s ¹	Network latency and high recurring cloud infrastructure costs ¹
-------------------------------	--	---	--	---------------------	--

Git worktrees provide the optimal sandboxing mechanism for desktop execution. A worktree creates a separate directory on an independent branch while sharing the parent repository's .git object store.

Below is the concrete implementation of a Worktree Sandbox Manager designed defensively for Windows:

TypeScript

```
import { SimpleGit, SimpleGit } from 'simple-git'
import { path } from 'path'
import { promises as fs } from 'fs/promises'
import { exec } from 'child_process'
import { promisify } from 'util'

const execAsync = promisify(exec)

export class WorktreeSandboxManager {
  private git: SimpleGit
  private rootRepoPath: string
  private worktreeBaseDir: string

  constructor(rootRepoPath: string) {
    this.rootRepoPath = path.resolve(rootRepoPath)
    this.worktreeBaseDir = path.join(this.rootRepoPath, '.swarm_worktrees')
    this.git = simpleGit(this.rootRepoPath)
  }

  public async initialize(): Promise<void> {
    await fs.mkdir(this.worktreeBaseDir, { recursive: true })
  }

  public async provisionWorkspace(agentId: string, baseBranch: string, 'main') Promise<
```

```

workspacePath string branch string :=
    const branch = `swarm/${agentId}-${Date.now()}`
    const workspacePath = path.join this.worktreeBaseDir, agentId)

    await this.defensiveCleanupDirectory(workspacePath)
    await this.gitRaw('worktree' 'add' '-b' branch, workspacePath, baseBranch))

    return workspacePath, branch }

}

public async teardownWorkspace(agentId string) Promise void {
    const workspacePath = path.join this.worktreeBaseDir, agentId)

    process.chdir this.rootRepoPath)

    try
        await this.gitRaw('worktree' 'remove' '--force' workspacePath))
    catch
        // Worktree command failed due to lingering file locks; fallback to manual cleanup

    await this.defensiveCleanupDirectory(workspacePath)
    await this.gitRaw('worktree' 'prune'

}

private async defensiveCleanupDirectory(dirPath string) Promise void {
    const delays = [50, 150, 500, 2000]
    for const delay of delays {
        try
            await fs.rm(dirPath, {recursive: true, force: true})
        return
        catch (err any)
            if (err.code === 'EBUSY' || err.code === 'EPERM')
                if (process.platform === 'win32')
                    try
                        await execAsync `powershell -Command "Get-Process | Where-Object { $_.Path -like
'${dirPath}*' } | Stop-Process -Force"`)
                    catch
                        // Ignore process query errors
                }
            }
        await new Promise (resolve) => setTimeout (resolve, delay)
        else if (err.code === 'ENOENT')

```

```
    return
  else
    throw err
  }
}
```

When integrating completed branches back into the main line, merge conflict detection must account for Git's conflict style configuration²². When repositories configure `merge.conflictStyle` to `diff3` or `zdiff3`, Git generates four boundary markers rather than the standard three, including a `|||||` ancestor marker²². Automated merge verification scripts must detect all four markers to prevent ancestor code blocks from silently leaking into the production codebase²².

Human-in-the-Loop Governance, Live Steering, and Time Travel

A critical vulnerability of autonomous agent swarms is hallucinatory divergence: an agent makes an erroneous design assumption early in execution, and subsequent actions compound that initial error¹. Maintaining control requires a defense-in-depth governance model that couples low-level command interception with durable state machine suspension and visual inspection².

Governance Layer	Interception Primitive	Execution Target	Operational Behavior
Layer 1: Deterministic Hooks	<code>.cursor/hooks.json</code> (<code>failClosed: true</code>) ²	Shell commands, file writes, terminal execution ²	Immediate synchronous block of high-risk actions (e.g., <code>rm -rf</code> , <code>git push --force</code>) ²
Layer 2: Graph Interrupts	<code>LangGraph.js</code> <code>interrupt()</code> / durable <code>checkpoint</code> ²³	High-level architectural transitions, deployment nodes ²	Graph halts, serializes execution state to SQLite, and awaits resume payload ²
Layer 3: Visual Inspection Surface	<code>@monaco-editor/react</code> <code>DiffEditor</code> [cite: 25, 26]	Code modifications, parameter	Human modifies code directly or provides natural

		payloads, handoff specs ²	language feedback ¹
--	--	---	--------------------------------

The orchestration layer integrates @langchain/langgraph to manage durable state suspension². When an execution path reaches a review gate, the node invokes interrupt()²⁴. This suspends execution by raising an internal exception caught by the runtime, serializing the full graph state to the checkpoint²⁴:

TypeScript

```
import { StateGraph, Annotation, interrupt, Command } from '@langchain/langgraph'
import { BaseCheckpointSaver } from '@langchain/langgraph-checkpoint'

export const SwarmState = Annotation.Root({
  taskPrompt: Annotation.string
  proposedChanges: Annotation.Array(path string diff string original string modified
string)
  humanReviewDecision: Annotation['APPROVED' | 'REJECTED' | 'EDITED']
  humanFeedback: Annotation.string
})

export function createHumanReviewGraph(checkpointer: BaseCheckpointSaver) {
  const workflow = new StateGraph(SwarmState)
  workflow.addNode('humanReviewGate', async (state) => {
    const decision = interrupt({
      message: 'Human review required for proposed code modifications.'
      files: state.proposedChanges
    })

    return {
      humanReviewDecision: decision.status,
      humanFeedback: decision.feedback ?? '',
      proposedChanges: decision.editedChanges ?? state.proposedChanges
    }
  })

  workflow.addConditionalEdges('humanReviewGate', (state) => {
    if (state.humanReviewDecision === 'APPROVED' || state.humanReviewDecision ===
'EDITED')
      return 'applyAndMergeNode'
  })
}
```

```

    return 'reworkNode'
  }

  return workflowCompleted ? checkPointer() : null
}

```

When an agent hits an interrupt, the user interface exposes three review modalities to the human operator¹. The first modality is binary approval, wherein the user verifies the proposed plan or diff and resumes execution without alterations via new `Command({ resume: { status: 'APPROVED' } })`¹. The second modality is rejection with corrective natural language guidance; the orchestrator resumes execution by routing the state back to the agent node while appending the reviewer's instructions to the context window, directing the model to attempt a refined pass¹. The third modality is direct payload mutation, allowing the human operator to modify code diffs or parameters directly in the visual interface before resuming¹. This eliminates secondary hallucinations and avoids consuming inference budget on minor syntax corrections¹. Visual code review and mutation are implemented using `@monaco-editor/react` with the `DiffEditor` component²⁵. The editor displays the original file alongside the agent's proposed modifications, allowing direct inline edits²⁵.

```

TypeScript
import React, { useRef } from 'react'
import { DiffEditor } from '@monaco-editor/react'

interface CodeReviewModalProps {
  originalCode: string
  modifiedCode: string
  language: string
  onApprove: (finalModifiedCode: string) => void
  onReject: (feedback: string) => void
}

export const CodeReviewModal: React.FC<CodeReviewModalProps> = ({
  originalCode,
  modifiedCode,
  language,
  onApprove,
  onReject,
}) => {
  const diffEditorRef = useRef<any>(null)
}

```

```

const handleEditorMount = (editor: any) => {
  diffEditorRef.current = editor
}

const handleConfirmApproval = () => {
  if (diffEditorRef.current) {
    const finalCode = diffEditorRef.current.getModifiedEditor().getValue()
    onApprove(finalCode)
  }
}

return (
  <div className="flex flex-col h-full bg-slate-900 border border-slate-700 rounded-lg p-4">
    <div className="flex justify-between items-center mb-3">
      <h3 className="text-white font-semibold text-lg">Inspect and Modify Agent
      Output</h3>
      <div className="flex gap-2">
        <button
          onClick={() => onReject('Approach requires revision, check naming and tests.')}
          className="px-4 py-1.5 bg-red-600 hover:bg-red-700 text-white rounded-
            text-sm
          >
            Reject with Feedback
        </button>
        <button
          onClick={handleConfirmApproval}
          className="px-4 py-1.5 bg-emerald-600 hover:bg-emerald-700 text-white
            rounded text-sm
          >
            Approve & Continue
        </button>
      </div>
    </div>
    <div className="flex-1 min-h-[400px]">
      <DiffEditor
        height="100%"
        language={language}
        original={originalCode}
        modified={modifiedCode}
        onMount={handleEditorMount}
        theme="vs-dark"
      />
    </div>
  </div>
)

```

```
options={
  renderSideBySide: true,
  readOnly: false,
  automaticLayout: true,
}
}
}
}
}
}
}
}
}
```

When an agent executes an erroneous chain of actions across multiple turns, standard conversational prompting to fix the problem wastes tokens and risks entrenching errors¹. Using LangGraph's checkpointer, the orchestrator retrieves the full history of checkpoints for a specific thread_id via graph.getStateHistory({ configurable: { thread_id } })²⁸. The user can select any prior checkpoint from the visual timeline, enabling state time travel²⁸. The orchestrator forks execution from that historical checkpoint, resets the Git worktree to the exact commit associated with that step, truncates downstream conversation turns, and resumes execution along an alternate path²⁸.

Phased Implementation Roadmap

To manage engineering complexity and maintain rapid velocity, development is organized into five sequential phases, establishing local Windows stability before packaging for cross-platform distribution³.

Phase	Milestone Focus	Core Deliverables	Technology Stack	Key Risk Mitigation Strategy
Phase 1	Local Canvas & Visual Editor	Interactive node graph, custom agent cards, dynamic handles, edge routing	Vite, React 19, TypeScript, @xyflow/react, Tailwind CSS	Maintain 60 FPS panning/zooming via Zustand decoupled state
Phase 2	Windows Desktop Shell & Isolation	Electron wrapper, secure IPC bridge, Git worktree isolation	Electron, Node.js 22, simple-git, node-pty [cite: 3, 5]	Resolve Windows EBUSY file locking via process termination

		engine		cascades
Phase 3	Cursor SDK Execution Engine	Topological DAG scheduler, agent lifecycle management, cancellation	@cursor/sdk, Kahn's Algorithm, Web Animations API	Ensure deterministic interruption of active runs via <code>run.cancel()</code> [cite: 3, 5]
Phase 4	Ingestion, MCP & HITL Review	File drag-and-drop, folder mounting, dynamic MCP nodes, Monaco DiffEditor	@modelcontextprotocol/sdk, @monaco-editor/react, LangGraph.js	Prevent context bloat using structured JSON handoff schemas
Phase 5	Persistence & Distribution	SQLite checkpoints, workflow export/import, cross-platform packaging	better-sqlite3, electron-builder (NSIS, DMG, AppImage)	Sanitize secrets on export; handle Win32 MAX_PATH path lengths

Milestone 1: Local Canvas and Visual Editor Prototyping

The project begins by scaffolding the visual composition canvas as a pure Vite React application³. Scaffolding starts with Vite, React 19, and Tailwind CSS to guarantee rapid hot-reloading and modular styling³. Integrating @xyflow/react establishes the visual viewport equipped with interactive pan-and-zoom controls, a minimap, and background grid snapping³. The custom node inventory includes the AgentNode for model configuration and prompt editing, the FileInputNode for dropped document ingestion, the MCPNode for external server definitions, and the ApprovalNode for human inspection gates³. Dynamic handle management is implemented within the AgentNode using useUpdateNodeInternals to ensure that edge anchors automatically recalibrate whenever tools or file attachments are modified³. A global Zustand store is wired to manage node additions, parameter mutations, and edge validation rules without inducing unnecessary DOM re-renders across the canvas³.

Milestone 2: Windows Desktop Shell and Workspace Isolation

The second phase embeds the visual editor inside an Electron desktop shell, establishing operating system bridges and defensive workspace sandboxing³. Scaffolding targets Electron with Node.js 22.13+ to support the runtime requirements of downstream dependencies³. Secure Inter-Process Communication (IPC) is implemented via

contextBridge.exposeInMainWorld, ensuring renderer isolation while exposing typed system channels³. The application manifest is configured with `<longPathAware>true</longPathAware>` to eliminate Win32 path length limitations during deep directory operations³. The WorktreeSandboxManager is integrated using simple-git, creating isolated worktree checkouts for parallel agent processes¹. Defensive teardown logic is implemented to address Windows EBUSY errors, systematically terminating child process trees with `taskkill /pid <PID> /T /F`, releasing working directory locks via `process.chdir()`, and executing exponential backoff retries during worktree removals³. Terminal interactions leverage Windows ConPTY via node-pty compiled against the Electron ABI³.

Milestone 3: Cursor TypeScript SDK Integration and DAG

Orchestration

The third phase implements the core orchestration logic, compiling the visual diagram into an executable DAG and running agents through `@cursor/sdk`³. The orchestration engine compiles visual node-edge structures into topological execution tiers using Kahn's algorithm, establishing deterministic parallel execution stages³. When execution initiates, workers in unblocked tiers initialize `@cursor/sdk` instances bound to their designated Git worktrees¹. The engine replaces default system prompts with user-defined instructions via `Agent.create({ systemPrompt })` and streams execution tokens back across IPC to update node status badges in real time³. Custom SVG edge components animate during active runs using the Web Animations API, providing visual feedback as tasks progress across the graph³. Process controls link UI cancellation actions directly to `run.cancel()`, ensuring immediate halting of network streams and background child processes upon operator command³.

Milestone 4: Multi-Modal Ingestion, MCP Tool Integration, and HITL Gates

The fourth milestone completes the tool integration and human oversight capabilities³. An HTML5 drag-and-drop listener on input nodes captures user files, copies them to an isolated project run directory, and formats their contents as markdown blocks or multimodal image structures³. Integrating `@modelcontextprotocol/sdk` enables the system to spawn and manage local stdio MCP servers, dynamically registering external database queries and web tools in `@cursor/sdk` agent options². Human-in-the-loop governance is established by compiling `@langchain/langgraph` with an `interrupt()` node²⁴. When an agent proposes file modifications, execution halts, persisting state to storage and rendering the `@monaco-editor/react` DiffEditor²⁴. The operator can approve the diff, reject it with natural language feedback, or mutate the source code directly within Monaco before unpausing the graph¹. Live steering connects an inline interface input to `run.steer(text)`, allowing mid-execution adjustments without process restarts³.

Milestone 5: State Persistence, Workflow Portability, and Distribution

The final milestone delivers full state persistence, export/import portability, and desktop installer builds³. Local persistence is implemented via better-sqlite3 operating in Write-Ahead

Logging (WAL) mode, guaranteeing high-performance non-blocking reads and durable transaction snapshots across system reboots³. Graph configurations, run histories, and state checkpoints are indexed systematically³. Workflow export serializes topologies to standardized JSON schemas, automatically sanitizing credentials and private tokens while registering required keys in an environment manifest³. Workflows and referenced assets are packaged into compressed .swarm archives that validate schema integrity and prompt importing users for required credentials³. Packaging automation utilizes electron-builder to generate signed Windows NSIS installers (.exe), with GitHub Actions workflows structured to compile native bundles for macOS (.dmg) and Linux (.AppImage) as platform expansion proceeds³.

Architectural Conclusions and Strategic Recommendations

Building a visual multi-agent swarm platform transforms autonomous coding from ad-hoc script execution into a disciplined, observable software engineering system¹.

The visual interface foundation is best served by React Flow (@xyflow/react), which balances declarative HTML/JSX node customization, dynamic multi-port handle management via useUpdateNodeInternals, and performant SVG edge animation without the engineering friction of low-level canvas hit-testing³. For the desktop shell, Electron represents the optimal runtime for Windows-first development while providing direct cross-platform portability³. It executes the official Cursor TypeScript SDK (@cursor/sdk) natively within Node.js, completely eliminating the dual-runtime complexity, packaging bloat, and IPC serialization bottlenecks inherent in bundling an external Python environment³.

The Cursor TypeScript SDK (@cursor/sdk) remains the decisive choice over the early-beta Python SDK³. It delivers harness-level system prompt replacement, dynamic in-flight run steering via run.steer, native MCP tool registration, deterministic cancellation via run.cancel(), and runs directly within Node.js without requiring external bridge servers³.

Process isolation at the filesystem layer is mandatory to prevent concurrent agents from corrupting the Git staging index or colliding during file edits¹. Deploying each agent to an isolated Git worktree preserves concurrency, ensures clean rollback points, and permits isolated test execution¹. On Windows, this architecture must operate defensively against mandatory file locking by terminating MCP process trees with taskkill prior to worktree removal, configuring extended path lengths (MAX_PATH), and implementing backoff retry loops for filesystem teardown³.

Human-in-the-loop oversight is made robust through durable execution suspension using LangGraph.js interrupt() primitives, which decouple human review from active process lifecycles². Coupling this state suspension with @monaco-editor/react DiffEditor allows operators to inspect code diffs, mutate payloads directly, or rewind graph execution to earlier checkpoints via time travel¹. By adhering to this phased implementation strategy, engineering teams can build a visual, production-grade agent swarm platform that balances intuitive user interactions with enterprise-grade execution stability¹.

Works cited

1. Multi-Agent Swarm Architecture.pdf
2. Building Cursor Agent Swarms.pdf
3. Visual Agent Swarm Architecture.pdf
4. Parallel AI Coding Agents: How to Run an Army of LLMs Without Git,
<https://viruchith.com/articles/parallel-ai-coding-agents-how-to-run-an-army-of-llms-without-git-chaos/>
5. Cursor TypeScript SDK | Cursor Documentation,
<https://cursor.com/docs/sdk/typescript>
6. Cursor SDK Officially Released: “Cursor” Inside Your Applications,
<https://ai-engineering-trend.medium.com/cursor-sdk-officially-released-bringing-ai-powered-coding-to-your-applications-2a9352e92cd1>
7. Introducing the Cursor Python SDK - #10 by miao_gu,
<https://forum.cursor.com/t/introducing-the-cursor-python-sdk/161367/10>
8. Quick Start - React Flow, <https://reactflow.dev/learn>
9. Using the Cursor SDK to Build a Project Documenter,
<https://www.raymondcamden.com/2026/08/22/using-the-cursor-sdk-to-build-a-project-documenter>
10. SDK Changelog | Cursor Docs, <https://cursor.com/docs/sdk/changelog>
11. Cursor Growth Logic | Wery,
<https://www.wery.ai/en/creations/0a7b6adnngth-cursor-growth-logic>
12. Model Context Protocol (MCP) - AI SDK,
<https://ai-sdk.dev/docs/ai-sdk-core/mcp-tools>
13. @scrypted/node-pty | Yarn,
<https://classic.yarnpkg.com/en/package/@scrypted/node-pty>
14. Build from Source | Daintree Docs, <https://daintree.org/docs/build-from-source>
15. mcp-framework vs TypeScript SDK,
<https://www.mcp-framework.com/docs/why-mcp-framework>
16. Model Context Protocol: TypeScript SDKs for the Agentic AI ecosystem,
<https://www.speakeasy.com/blog/release-model-context-protocol>
17. cursor/sdk-bridge - GitHub, <https://github.com/cursor/sdk-bridge>
18. Cursor SDK Tutorial: Run Coding Agents With TypeScript - DataCamp,
<https://www.datacamp.com/tutorial/cursor-sdk>
19. @modelcontextprotocol/sdk - npm,
<https://www.npmjs.com/package/@modelcontextprotocol/sdk>
20. Releases · modelcontextprotocol/typescript-sdk - GitHub,
<https://github.com/modelcontextprotocol/typescript-sdk/releases>
21. Worktrees - ChatGPT Learn,
<https://learn.chatgpt.com/docs/environments/git-worktrees>
22. Scripted git conflict resolution silently keeps the ancestor section,
<https://agents.stackoverflow.com/t/ils/44ce169b-068e-45d0-a15d-500166400d24?tag=git&sort=trust&page=1>
23. @langchain/langgraph-checkpoint - NPM,
<https://www.npmjs.com/package/@langchain/langgraph-checkpoint?activeTab=d>

[ependents](#)

24. Interrupts - Docs by LangChain,
<https://docs.langchain.com/oss/javascript/langgraph/interrupts>
25. react-monaco-editor - NPM,
<https://www.npmjs.com/package/react-monaco-editor?activeTab=readme>
26. GitHub - suren-atoyan/monaco-react: Monaco Editor for React,
<https://github.com/suren-atoyan/monaco-react>
27. Implement DiffNavigator · Issue #84 · react-monaco-editor ... - GitHub,
<https://github.com/react-monaco-editor/react-monaco-editor/issues/84>
28. Checkpointers - Docs by LangChain,
<https://docs.langchain.com/oss/javascript/langgraph/checkpointers>
29. How to time travel to before interrupt and resume with a different,
<https://forum.langchain.com/t/how-to-time-travel-to-before-interrupt-and-resume-with-a-different-value/2434>